HospedaFlow - Sistema de Gestión Hotelera

Documento Guía y Resumen Técnico para Defensa de Proyecto

Asignatura: Desarrollo de Sistemas Web / Ingeniería de Software • **Sprint:** Sprint 1 (Etapa 3: Ejecución)
Rol de Defensa: **Desarrollador 4 (Frontend UI/UX & Asynchronous Frontend Developer)**
Arquitectura: Spring Boot REST API + Jackson JSON Persistence + Vanilla JS / Bootstrap 5 SPA

1. Mapeo de Historias de Usuario (Etapa 3 - Sprint 1) en GitHub

El tablero del proyecto en Trello organiza la **Etapa 3: Ejecución (Sprint 1)** en 15 historias de usuario fundamentales para el core del hotel (Habitaciones, Reservas, Perfiles y Ofertas). A continuación se documenta la correspondencia exacta entre cada historia y los componentes desarrollados en el repositorio de GitHub:

1.1 Módulo Habitaciones

Historia Trello	Frontend (Vista y JS)	Controlador REST (API)	Servicio y Persistencia
Historia Crear Habitaciones	`habitaciones.html` `formHabitacion` → `guardarHabitacion()` en `habitaciones.js`	**POST** `/api/habitaciones` `HabitacionRestController.crear()` (Recibe `HabitacionRequestDTO`)	`HabitacionService.crearHabitacion()` Valida número único y persiste en `habitaciones.js` vía `HabitacionJsonRepository`.
Historia Consulta de Disponibilidad de habitaciones	`habitaciones.html` (filtro por estado) y selector dinámico en `reservas.html`· `cargarHabitaciones()`	**GET** `/api/habitaciones?disponibles=true` `HabitacionRestController.listar()`	`HabitacionService.consultarDisponibilida` Filtra aquellas habitaciones con `EstadoHabitacion.DISPONIBLE`.
Historia eliminar Habitaciones	`habitaciones.html` Botón "Eliminar" en tabla → `eliminarHabitacion()` en `habitaciones.js`	**DELETE** `/api/habitaciones/{id}` `HabitacionRestController.eliminar()`	`HabitacionService.eliminarHabitacion()` Elimina la entidad por identificador numérico en el repositorio JSON.

1.2 Módulo Reservas

Historia Trello	Frontend (Vista y JS)	Controlador REST (API)	Servicio y Persistencia
Historia Registro de reserva de habitacion	`reservas.html` `formReserva` → `guardarReserva()` en `reservas.js`	**POST** `/api/reservas` `ReservaRestController.registrar()` (Recibe `ReservaRequestDTO`)	`ReservaService.registrarReserva()` Asigna estado `PENDIENTE`, valida fechas, resuelve huésped/habitación y guarda en `reservas.json`.
Historia Asignar Habitacion	`reservas.html` Selector `resHabitacion` o acción en fila → `reservas.js`	**PUT** `/api/reservas/{id}/habitacion/{habId}` `ReservaRestController.asignarHabitacion()`	`ReservaService.asignarHabitacion()` Vincula la habitación y cambia su estado a `OCUPADA`.
Historia Modificacion de fechas de reserva	`reservas.html` Botón "Modificar fechas" → modal/ prompt → `modificarFechas()`	**PUT** `/api/reservas/{id}/fechas` `ReservaRestController.modificarFechas()`	`ReservaService.modificarFechas()` Valida que `fechaEntrada < fechaSalida` y recalcula el monto final total.
Historia Confirmar reserva de habitacion	`reservas.html` Botón "Confirmar" → `confirmarReserva()` en `reservas.js`	**PUT** `/api/reservas/{id}/confirmar` `ReservaRestController.confirmar()`	`ReservaService.confirmarReserva()` Transiciona el estado de la reserva de `PENDIENTE` a `CONFIRMADA`.
Historia Cancelacion de reserva de habitacion	`reservas.html` Botón "Cancelar" → `cancelarReserva()` en `reservas.js`	**PUT** `/api/reservas/{id}/cancelar` `ReservaRestController.cancelar()`	`ReservaService.cancelarReserva()` Establece estado `CANCELADA` y libera la habitación vinculada a `DISPONIBLE`.

1.3 Módulo Perfiles y Control de Acceso

Historia Trello	Frontend (Vista y JS)	Controlador REST (API)	Servicio y Persistencia
Historia Crear Perfil / Creacion de perfil	`index.html` (Huéspedes) y `personal.html` (Recepcionistas). `perfiles.js`	**POST** `/api/usuarios` `PerfilRestController.crear()` (Recibe `UsuarioRequestDTO`)	`PerfilService.crearPerfilUsuario()` Instancia la subclase concreta (`Huesped`, `Recepcionista`, `Gerente`) y almacena en `usuarios.json`.
Historia Asignar rol	`personal.html` y formularios de registro. `perfiles.js`	**PUT** `/api/usuarios/{id}/rol` `PerfilRestController.asignarRol()`	`PerfilService.asignarRol()` Actualiza el enum `RolUsuario` del usuario respetando el modelo jerárquico.
Historia Consultar perfiles	`personal.html` Tabla de recepcionistas activos → `cargarRecepcionistas()`	**GET** `/api/usuarios` `PerfilRestController.listar()`	`PerfilService.consultarUsuarios()` Retorna el listado de usuarios con contraseñas filtradas por seguridad.
Historia Borrar perfil	`personal.html` Botón "Eliminar" en tabla → `eliminarRecepcionista()`	**DELETE** `/api/usuarios/{id}` `PerfilRestController.eliminar()`	`PerfilService.eliminarPerfil()` Remueve al usuario por ID del repositorio `UsuarioJsonRepository`.

1.4 Módulo Ofertas

Historia Trello	Frontend (Vista y JS)	Controlador REST (API)	Servicio y Persistencia
Historia Crear Oferta	`ofertas.html` `formOferta` → `guardarOferta()` en `ofertas.js`	**POST** `/api/ofertas` `OfertaRestController.crear()` (Recibe `OfertaRequestDTO`)	`OfertaService.crearOferta()` Valida porcentaje y vigencia de fechas; persiste en `ofertas.json`.
Historia Modificar Oferta	`ofertas.html` Edición rápida o formulario → `editarOferta()`	**PUT** `/api/ofertas/{id}` `OfertaRestController.modificar()`	`OfertaService.modificarOferta()` Actualiza porcentaje de descuento, fechas de validez o estado activo.
Historia Eliminar Oferta	`ofertas.html` Botón "Eliminar" en tarjeta → `eliminarOferta()`	**DELETE** `/api/ofertas/{id}` `OfertaRestController.eliminar()`	`OfertaService.eliminarOferta()` Remueve la oferta de la persistencia JSON.

2. Explicación del Flujo del Código (Arquitectura y Ciclo de Vida)

HospedaFlow adopta una **Arquitectura en Capas estricta (Separation of Concerns)** bajo el patrón **MVC Desacoplado con API REST**. El Frontend y el Backend están completamente separados y se comunican exclusivamente mediante el protocolo HTTP con cargas útiles en formato JSON estándar.

2.1 Las 4 Capas del Sistema

Capa 1: Cliente / Frontend SPA Asíncrono (HTML5, Bootstrap 5, Vanilla JS)

Construida con JavaScript nativo moderno (`ES6+`) sin frameworks pesados. Todo el intercambio de información con el servidor se realiza de manera **asíncrona** mediante `async/await` y la API estándar `fetch()` centralizada en el helper `apiFetch()` en `comun.js`. La interfaz **nunca recarga la página por completo**: intercepta eventos con `evento.preventDefault()` y actualiza únicamente los fragmentos del DOM necesarios (tablas, alertas flotantes, contadores), brindando una experiencia de usuario rápida y moderna.

Capa 2: Controladores REST (Spring Boot RestControllers)

Punto de entrada de las peticiones HTTP (`@RestController`, `@RequestMapping`). **No contienen lógica de negocio ni cálculos matemáticos**. Su única responsabilidad es recibir el JSON de la solicitud, deserializarlo en objetos de transferencia de datos (**DTOs** como `ReservaRequestDTO`, `HabitacionRequestDTO`), invocar al servicio correspondiente y retornar la respuesta empaquetada en un `ResponseEntity` con el código HTTP semántico correspondiente (`200 OK`, `201 CREATED`, `400 BAD REQUEST`).

Capa 3: Lógica de Negocio y Dominio (Spring Services)

Clases anotadas con `@Service` inyectadas con `@Autowired`. Es el corazón del sistema donde residen las reglas de negocio del hotel: validación de unicidad de habitaciones, control de estados (`DISPONIBLE`, `OCUPADA`, `MANTENIMIENTO`), verificación de solapamiento de fechas, aplicación de descuentos de ofertas vigentes y transiciones de estados de reservas (`PENDIENTE` → `CONFIRMADA` o `CANCELADA`).

Capa 4: Persistencia Ligera en Archivos JSON (JsonRepositories)

En cumplimiento de los requerimientos de persistencia ligera sin bases de datos SQL pesadas, el sistema implementa repositorios basados en Jackson (`ObjectMapper`). Los repositorios leen y escriben atómicamente en colecciones de objetos serializadas en archivos JSON (`habitaciones.json`, `reservas.json`, `usuarios.json`, `ofertas.json`), asignando llaves primarias autoincrementales de forma determinista.

2.2 Ciclo de Vida Paso a Paso de una Petición (Ejemplo: Registrar Reserva)

1. **Acción del Usuario en Frontend:** El usuario completa el formulario en `reservas.html` seleccionando fecha de llegada, fecha de salida, habitación y oferta, y hace clic en "Guardar reserva".
2. **Captura del Evento JS:** `reservas.js` intercepta el submit con `evento.preventDefault()`, realiza validaciones locales (campos obligatorios) y empaqueta el cuerpo JSON correspondiente.
3. **Llamada Asíncrona (apiFetch):** Se dispara un `fetch('http://localhost:8080/api/reservas', { method: 'POST', body: ... })` con cabeceras `Content-Type: application/json`.
4. **Enrutamiento y DTO en Backend:** Spring Boot enruta la solicitud al método `ReservaRestController.registrar()`. Jackson convierte automáticamente el JSON en una instancia de `ReservaRequestDTO`.
5. **Ejecución de Lógica en Servicio:** El controlador delega a `ReservaService.registrarReserva()`, el cual verifica que el huésped exista, que la habitación esté disponible y que la fecha de salida sea posterior a la de entrada.
6. **Persistencia en Disco:** `ReservaJsonRepository` lee `reservas.json`, calcula el siguiente ID secuencial, añade la nueva reserva a la lista y reescribe el archivo JSON con formato indentado.
7. **Respuesta HTTP:** El controlador responde con estado `201 CREATED` y el cuerpo JSON de la entidad creada.
8. **Actualización Dinámica del DOM:** La promesa en `reservas.js` se resuelve con éxito, invoca a `mostrarAlerta('Reserva creada con éxito', 'success')`, limpia el formulario y vuelve a consultar la tabla de reservas en segundo plano sin refrescar la página.

2.3 Mecanismos Transversales Clave para la Defensa

> **1. Manejo Centralizado de Excepciones (GlobalExceptionHandler.java)**
> Cualquier error de negocio (por ejemplo, intentar reservar con fechas inválidas o registrar una habitación duplicada) dispara una excepción `IllegalArgumentException` o `IllegalStateException` en el servicio. El componente `GlobalExceptionHandler` captura la excepción con `@ExceptionHandler` y devuelve un código HTTP `400 Bad Request` con el mensaje amigable en formato JSON. Esto impide que el servidor colapse o devuelva errores 500 no controlados.

> **2. Seguridad y Control de Acceso Basado en Roles (RBAC)**
> En el Frontend, el módulo `comun.js` gestiona la sesión mediante `sessionStorage`. Cada vista invoca a `protegerPagina([ROLES.GERENTE, ...])`: si el usuario no tiene una sesión activa o no posee el rol necesario, es redirigido inmediatamente a `index.html`. Adicionalmente, la función `renderNavbar()` construye dinámicamente los menús según el rol autenticado (Gerente tiene acceso a Personal y Habitaciones completas; Recepcionista solo a Habitaciones en modo consulta y Reservas).

> **3. Protección de Credenciales (JacksonConfig.java)**
> Para evitar fugas de información sensible, la configuración de serialización de Jackson está configurada para excluir la contraseña (`password` / `contrasena`) de las respuestas JSON generadas hacia el cliente al consultar usuarios, protegiendo los datos de los empleados y huéspedes.

3. Preguntas Frecuentes de la Defensa y Cómo Responderlas

¿Por qué se usan DTOs en lugar de recibir directamente las entidades en los controladores?

Respuesta: Porque los DTOs (*Data Transfer Objects*) aíslan el contrato de la API del modelo interno de persistencia. Permiten recibir IDs foráneos sencillos (como `huespedId` o `habitacionId`) desde el cliente en lugar de objetos anidados complejos, mejorando la seguridad, evitando ciclos de serialización infinita y permitiendo validar entradas de forma independiente.

¿Cómo garantiza el Frontend que la aplicación sea asíncrona?

Respuesta: Al no utilizar recargas de página tradicionales (envíos síncronos de formularios HTML). Se intercepta cada acción con escuchadores de eventos JavaScript (`addEventListener`), se ejecutan llamadas HTTP en segundo plano con `fetch` y `async/await`, y se manipula el árbol DOM de forma reactiva en base a la respuesta obtenida.

¿Dónde se validan las reglas de negocio, en el cliente o en el servidor?

Respuesta: Se implementa **validación en doble capa**. En el cliente se realizan validaciones preliminares de UX (campos requeridos, formato mínimo de contraseña) para retroalimentar al usuario al instante. Sin embargo, **la validación autoritativa e inmutable reside en los Servicios de Spring Boot** (`HabitacionService`, `ReservaService`), garantizando la integridad de los datos incluso si se realizan peticiones externas a la API.